

NYCU – ECE DCS – 2026

Final Project

Design: Conformer Accelerator

Data Preparation

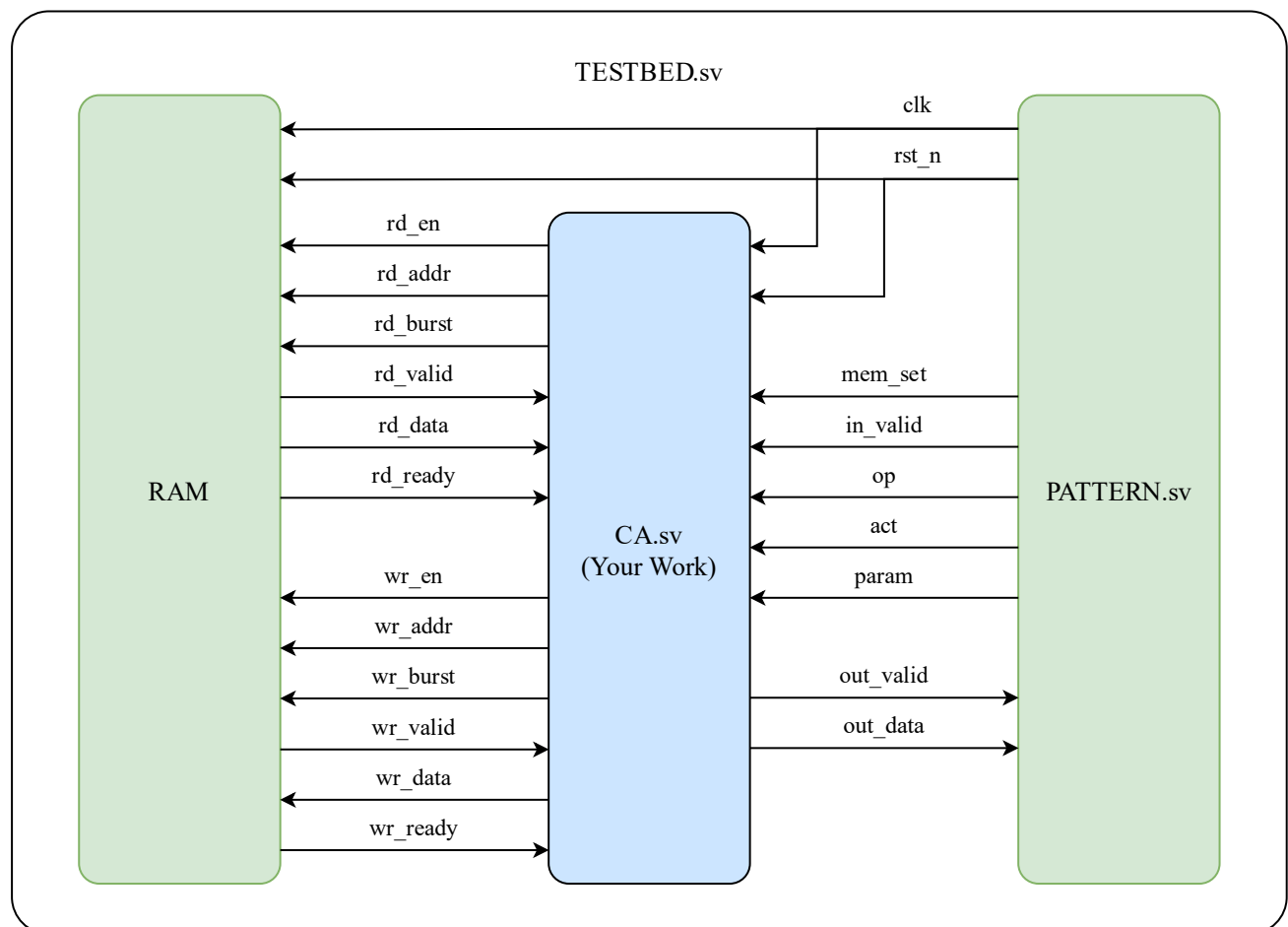
1. Extract the files needed from TA's directory:

```
$ tar -xvf ~es26dcsTA01/Final.tar
```

2. The extracted files contain:

```
Final/  
├── 00_TESTBED/  
├── 01_RTL/  
├── 02_SYN/  
├── 03_GATE/  
└── 09_UPLOAD/
```

Block Diagram



Background

The Conformer model is a hybrid neural network architecture that integrates the strengths of both Transformers and Convolutional Neural Networks (CNNs). By combining the Transformer's ability to capture long-range dependencies with the CNN's proficiency in local feature extraction, the model achieves superior performance in Automatic Speech Recognition (ASR). The figure below shows the typical architecture of Conformer model.

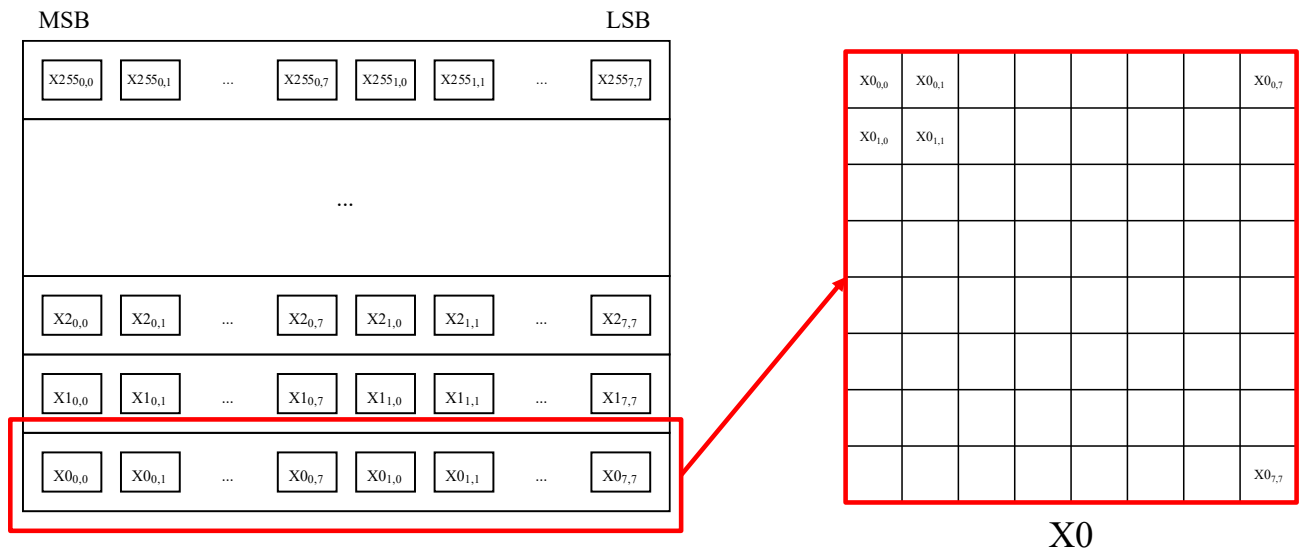
Design Description

In Final Project, you are asked to implement a Conformer Accelerator, which load 4-bit 8*8 data array from RAM and perform specified computation and activation function, then quantize the array back to 4-bit. After the operations, the Accelerator outputs last token as output and store the resulting array back to RAM.

- RAM:

To optimize power consumption and minimize circuit area, SRAM is utilized instead of D-Flip-Flops (DFFs) for high-capacity data storage. In this final project, we implemented a Pseudo-RAM provided by the TA to store input data arrays. The following figure illustrates the data arrangement within the RAM.

- Each word of RAM is a 8*8 data matrix (each data is signed 4-bit).



- Operations:

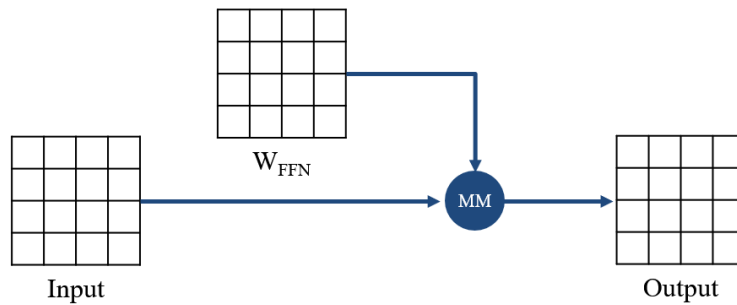
Your design is required to perform operation sets specified by the pattern. Each operation set contains 1 computation and 1 activation function. Below are the types of computation and activation function.

[1:0] op	Computation Types
2'b00	FFN
2'b01	Convolution
2'b10	Single-Head Attention
2'b11	Multi-Head Attention

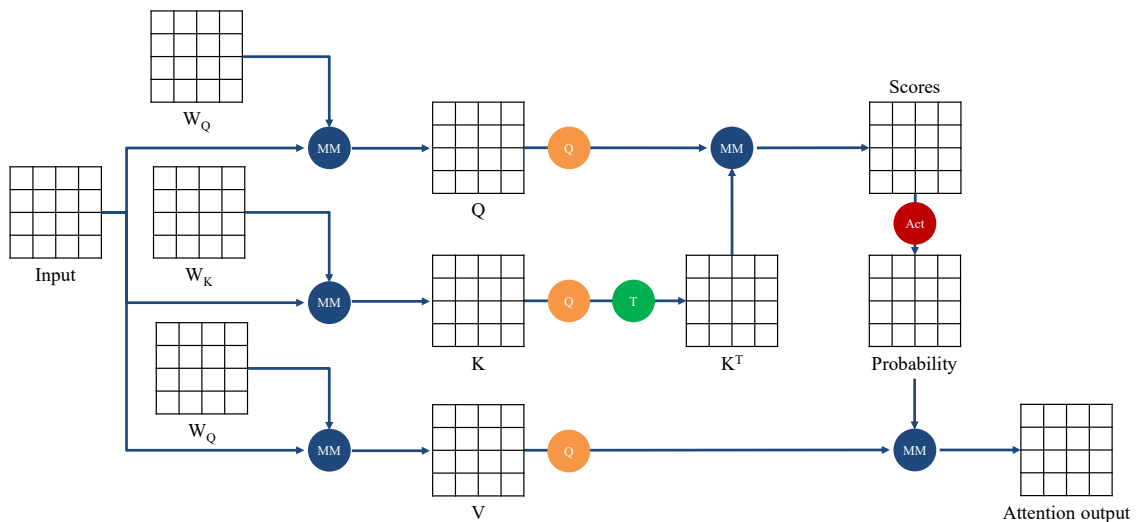
[1:0] act	Activation Functions
2'b00	ReLU
2'b01	RAT
2'b10	CAT
2'b11	BAT

■ Computation Types

1. **FFN (Linear):** It performs a matrix multiplication $y = Wx$. Every input feature is connected to every output feature via a unique learned weight.



2. **Convolution:** A small "window" or kernel (a matrix of weights) slides across the input data. At each step, it computes the dot product between the kernel and the local patch of the input it is hovering over.
3. **Single-Head Attention:** It computes a dynamic set of weights using Queries (Q), Keys (K), and Values (V). The model compares a query against all keys to generate "attention scores". These scores determine how much each corresponding Value should contribute to the final output.

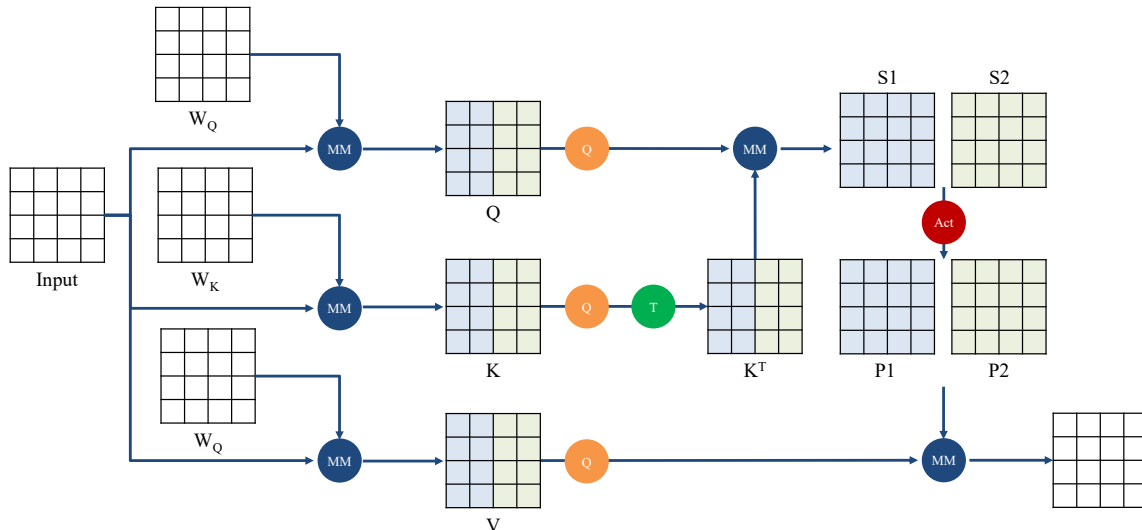


*MM: matrix multiplication Q: quantization T: transpose Act: activation

*Note that Act in attention computation is the same with HW04, which is

$$act(x) = \begin{cases} x, & x \geq 0 \\ x/4 \text{ (truncate toward zero)}, & \text{otherwise} \end{cases}$$

4. **Multi-Head Attention:** Similar to single-head attention, but here we split Q and K along the feature column dimension into Q1/Q2 and K1/K2. Then compute the attention score matrices for each head, like the two-head mode in HW04.



■ Activation Types

1. **ReLU:** It acts as a simple filter with a single rule: if the input value is positive, it passes it through exactly as it is. If the input value is negative (or zero), it outputs zero.
2. **RAT (Row-wise Average Threshold):** An activation function that processes each row of a matrix individually. It first computes the average of the row (by summing all elements, then dividing by the count), which serves as the threshold. Values greater than or equal to this threshold are kept; others are divided by 8.
3. **CAT (Column-wise Average Threshold):** Similar to RAT, but it operates on each column of a matrix individually.
4. **BAT (Block Average Threshold):** Similar to RAT, but operates on each block of a matrix (block size = 4*4).

- Quantization:

Following the computation and activation function, the design must quantize the data array to a 4-bit format using Power-of-Two (PoT) quantization. By utilizing the power-of-two value nearest to the maximum as a scaling factor, this mechanism remains both dynamic and hardware-efficient.

For more information about computation flow, operations and RAM, see [2026_DCS_FP_appendix.pdf](#).

Inputs

Signal name	Number of bit	Description
clk	1	Clock signal
rst_n	1	Asynchronous active-low reset signal
Mem_set	1	Indices that RAM is properly set and ready for Read or Write operation
in_valid	1	Pulled high during operation set input
op	2	Test computation type 2'b00: FFN 2'b01: Convolution 2'b01: Single-Head Attention 2'b11: Multi-Head Attention
act	2	Test activation function 2'b00: ReLU 2'b01: RAT 2'b01: CAT 2'b11: BAT
param	256	Parameters for test computation
RAM Signals		
rd_valid	1	Read data valid. High when rd_data is valid.
rd_data	256	Read data.
rd_ready	1	High when RAM is prepared to accept a new read request.
wr_valid	1	Write data valid. High when wr_data is valid.
wr_ready	1	High when RAM is prepared to accept a new write request.

Outputs

Signal name	Number of bit	Description
out_valid	1	Pulled high during output (reset required)
out_data	32	Last token of the result matrix (reset required)
RAM Signals		
rd_en	1	Read enable. Assert high to initiates a read transaction.
rd_addr	8	Read address. The starting address for the read operation.
rd_burst	3	Burst length. Read 2^{rd_burst} for each read operation.
wr_en	1	Write enable. Assert high to initiates a write transaction.
wr_addr	8	Write address. The starting address for the write operation.
wr_burst	3	Burst length. Write 2^{rd_burst} for each write operation.
wr_data	256	Write data.

Specifications

1. Top module name: CA(File name : CA.sv)
2. After asynchronous active-low reset, all output signals to PATTERN must be reset to zero.
3. Each Output must be produced within 10000 cycles after in_valid pulls low or last out_valid pulls low.
4. The 02_SYN result **must not have any errors** and **must not contain any latches**.
5. The slack in the timing report must be non-negative and the result must be "Met".
6. Input delay = 0.5 * clock period; Output delay = 0.5 * clock period.
7. Do not use *error*, *latch*, *congratulation* or *fail* as names for logic / wire / reg / submodule / parameter , otherwise the demo result will be fail.

Note: * represents any symbol before or after the word. Ex: error_test is prohibited.

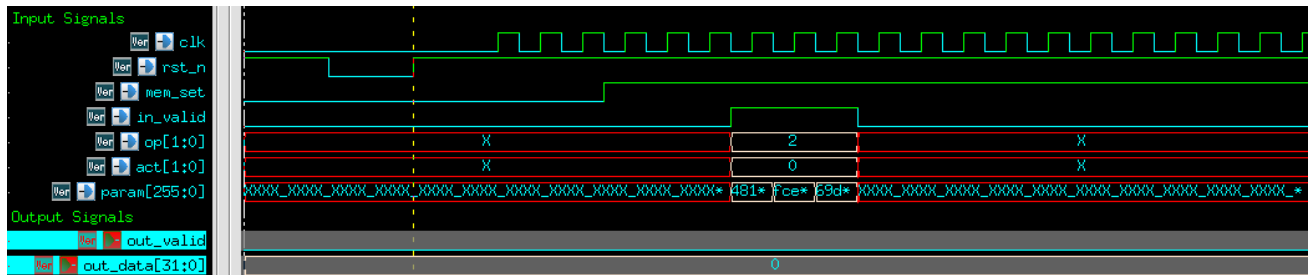
8. **You can use for loops** in this homework.
9. The demonstration will use **a hidden test pattern**. Ensure your design can handle various random traffic conditions, not just the test cases provided.
10. In final project, the clock cycle is adjustable.

```
`define CYCLE_TIME 10.0 (00_TESTBED / PATTERN.sv: line 1)
```

```
set CYCLE 10.0 (02_SYN / syn.tcl: line 11)
```

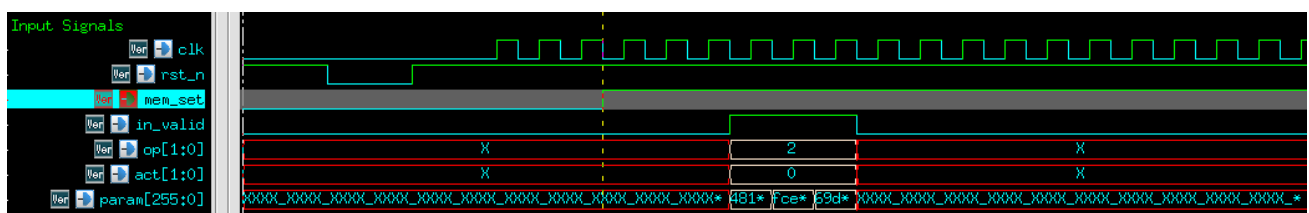
Example waveform

1. After reset signal, output signals to pattern should be reset to zero.

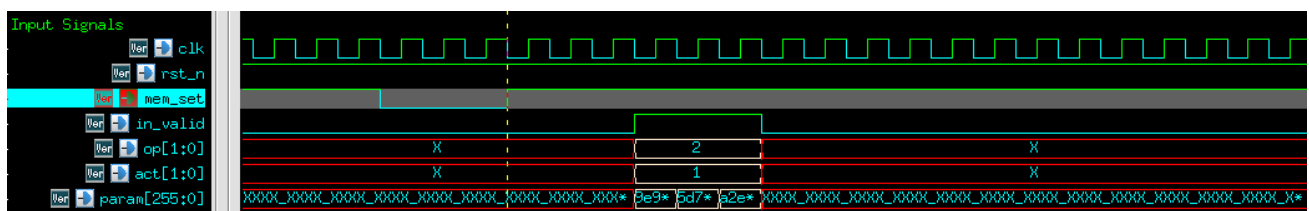


2. Signal `mem_set` will be pulled high after RAM is set.

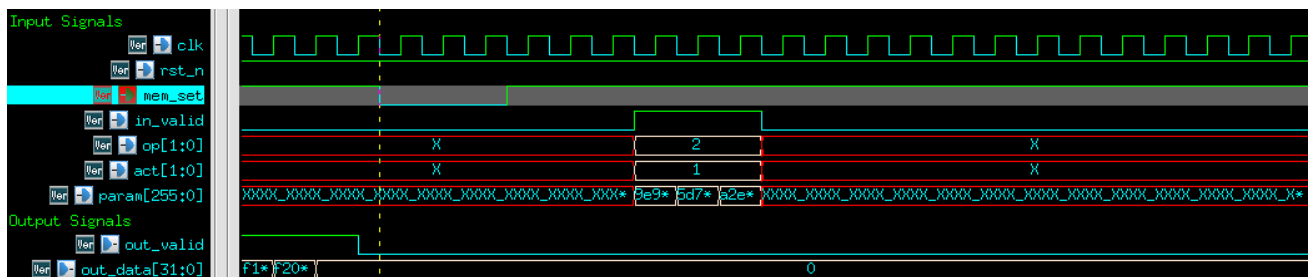
(1) First RAM (beginning of pattern):



(2) Switch to new RAM:

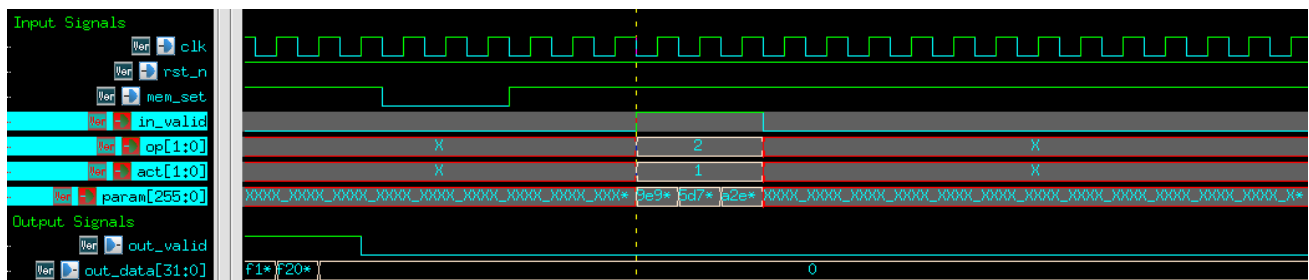


3. Signal `mem_set` will be pulled low when all operation set of a RAM is completed.

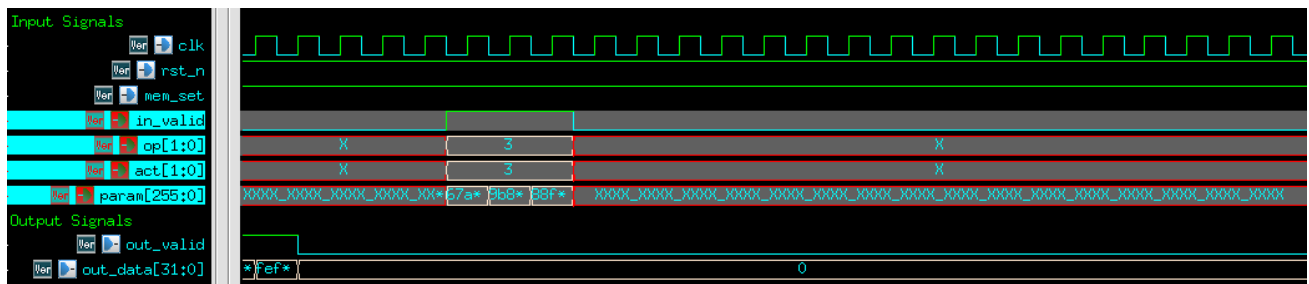


4. Operation set will be specified after a new RAM is set, or 256 data arrays completed.

(1) New RAM:

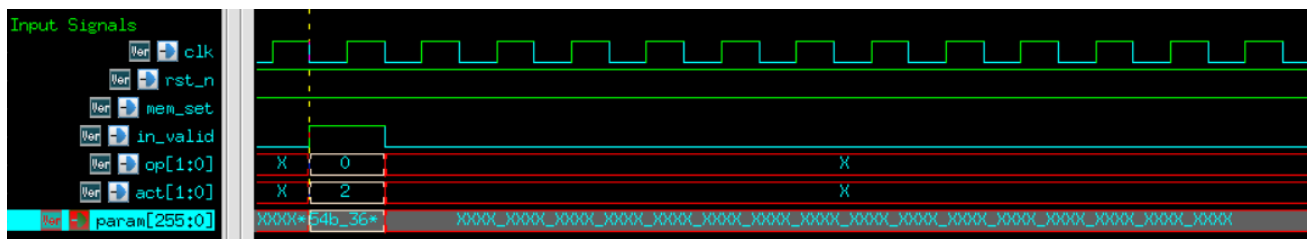


(2) 256 data arrays completed, give new operation set for same RAM data:

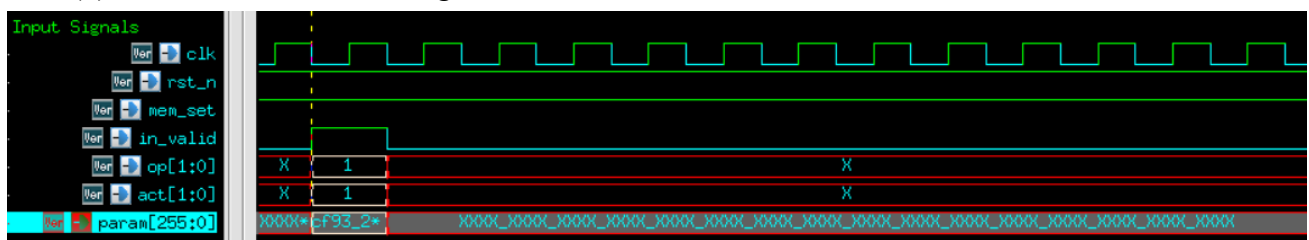


5. Parameters of corresponding computation:

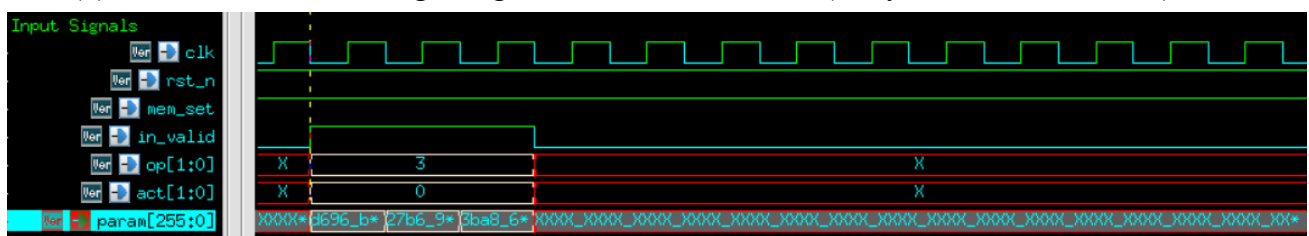
(1) FFN: Weight is given in raster scan order from MSB to LSB.



(2) Convolution: Kernel is given in raster scan order from MSB to LSB.

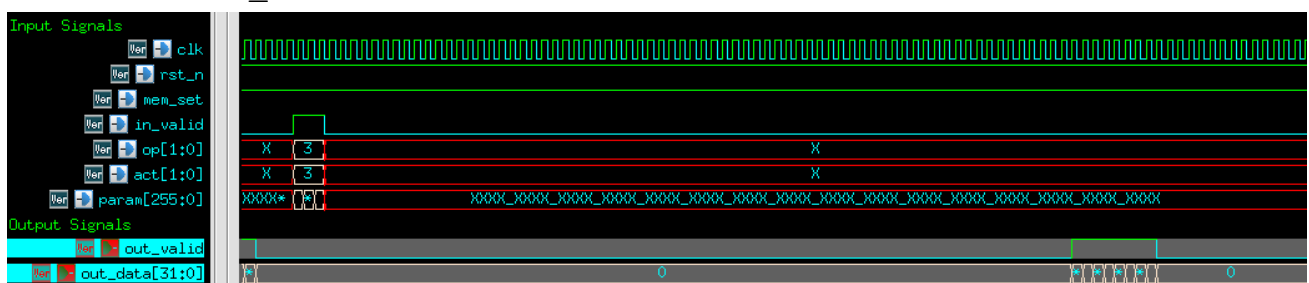


(3) Attention: Q/K/V weight is given in raster scan order (1st cycle: Q, 2nd: K, 3rd: V).

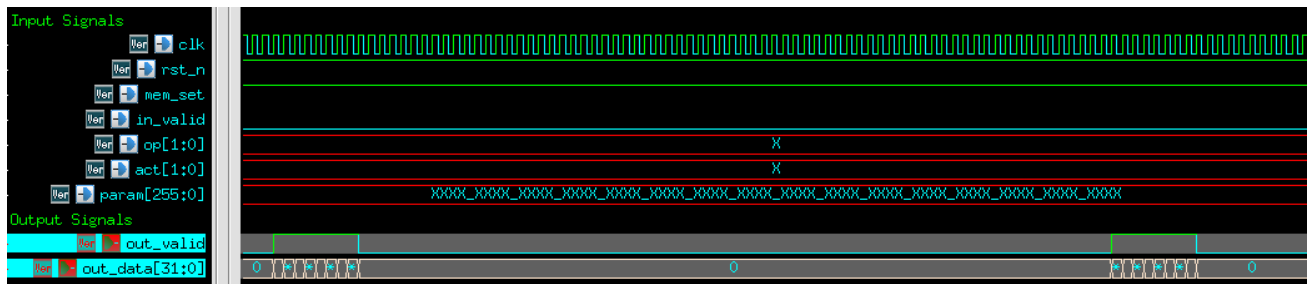


6. Output must be produced within 10000 cycles after in_valid pulls low or last out_valid pulls low.

(1) After in_valid:

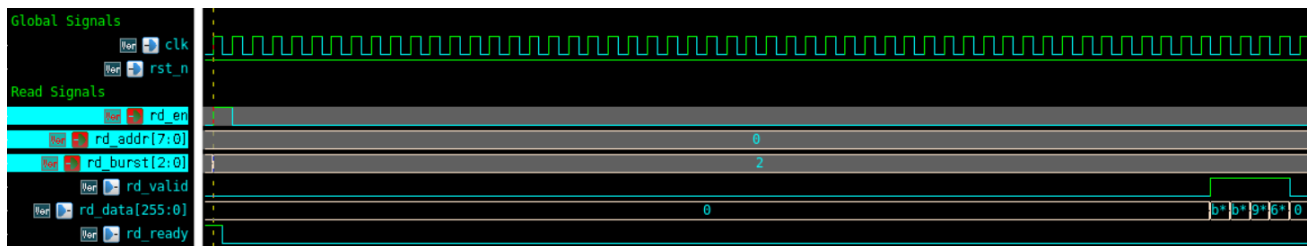


(2) After out_valid:

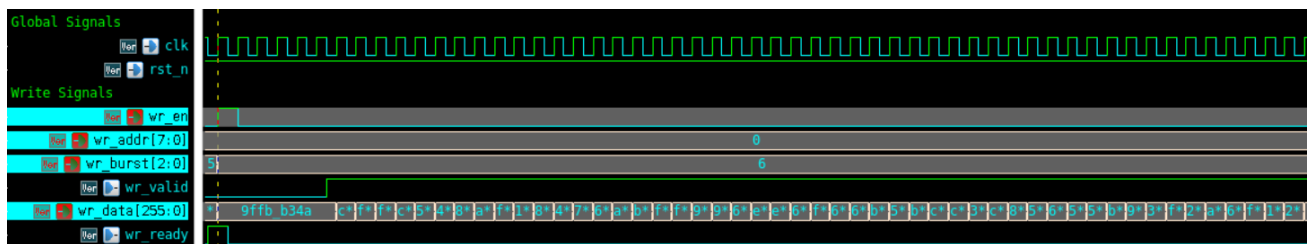


7. RAM:

- (1) Read Operation: When `rd_ready` is high, pull `rd_en` to initial a Read Operation. You should provide `rd_addr`, `rd_burst` at the same time.



- (2) Write Operation: When `wr_ready` is high, pull `wr_en` to initial a Read Operation. You should provide `wr_addr`, `wr_burst` at the same time. The first data will be written into RAM at rising edge of `wr_valid`. Then, you can alternate `wr_data` to complete burst write.



Upload file

1. Upload the following files:

File	Filename	
SystemVerilog Code		Upload using ./01_upload under 09_UPLOAD/
Report	final_report_dcsXXX.pdf (XXX is server account)	Upload to E3
PATTERN (optional)	PATTERN_dcsXXX.sv	Upload to E3

* Note that the script requires time to synchronize with the NTP server. Please avoid last-minute submissions.

* Incorrect filenames will result in a 5-point deduction.

2. Submission time:

Submit	Deadline
1de	2026/06/11 23:59
2de	2026/06/18 23:59

Grading policy

Criteria	%	Description
Pass the RTL, Synthesis and GATE level simulation	60	Pass all pattern and no error in synthesis.
Performance	25	Ranking formula: execution_cycles * cycle_time * area
Report	15	The report should be concise, not exceeding four A4.
PATTERN	5	Bonus (Optional)

* $\text{execution_cycles} = (\text{cycles between in_valid falls and out_valid rises}) + (\text{cycles between out_valid falls and out_valid rises} + 1)$

* For 2de, total score will get **30% off**.

Note

Template folders and reference commands:

- 01_RTL/ (RTL simulation) → **./01_run_vcs_rtl**
- 02_SYN/ (synthesis) → **./01_run_dc**
- 03_GATE/ (GATE simulation) → **./01_run_vcs_gate**
- 09_UPLOAD/ (upload) → **./01_upload**